TOUR MANAGEMENT SYSTEM

Database Management Systems Lab

Project Report

Submitted By :

Name: Md. Motiur Rahman

Student ID: C243119

Department: Computer Science & Engineering (CSE)

Semester: 4th Semester

International Islamic University Chittagong (IIUC)

Submitted To :

Mizanur Rahman

Course Teacher & Project Advisor

Department of Computer Science & Engineering

International Islamic University Chittagong (IIUC)

Course Information :

Course Title: Database Management Systems Lab

Course Code: CSE 2424

Submission Date :

Jun 31, 2026

Certificate :

CERTIFICATE OF APPROVAL :

This is to certify that the project entitled **"Tour Management System"** has been successfully completed by **Md. Motiur Rahman (Student ID: C243119)** as a partial fulfillment of the requirements for the course **Database Management Systems Lab (CSE 2424)** in the Department of Computer Science and Engineering, International Islamic University Chittagong (IIUC).

The project has been developed using **C++** as the frontend programming language and **MySQL** as the backend database management system. During the development process, the student implemented various database concepts including relational database design, normalization, primary and foreign keys, SQL queries, views, triggers, stored procedures, indexing, and database connectivity.

To the best of my knowledge, this project is an original work completed under my supervision and is suitable for submission as a DBMS Lab project.

Advisor :

Mizanur Rahman

Course Teacher

Database Management Systems Lab (CSE 2424)

Department of Computer Science & Engineering

International Islamic University Chittagong (IIUC)

Signature: _Mizanur Rahman_____

Date: __jun 31, 2026_____

Acknowledgement

ACKNOWLEDGEMENT :

First and foremost, I express my sincere gratitude to Almighty Allah for giving me the strength, patience, and guidance to complete this project successfully.

I would like to express my heartfelt appreciation to my respected course teacher and project advisor, **Mr. Mizanur Rahman**, for his valuable guidance, constructive suggestions, and continuous encouragement throughout the development of this project. His advice and support played a significant role in improving both the design and implementation of the system.

I am also thankful to the Department of Computer Science and Engineering, International Islamic University Chittagong (IIUC), for providing the opportunity to complete this Database Management Systems Lab project.

Finally, I would like to thank my family, friends, and classmates for their continuous motivation and support during the completion of this project.

Abstract

ABSTRACT :

The **Tour Management System** is a database-driven application designed to simplify and automate the overall tour management process. Traditional tour booking systems often rely on manual record keeping, making it difficult to manage customer information, bookings, payments, hotels, transportation, and reports efficiently. This project addresses these challenges by providing a centralized and well-structured database management solution.

The system has been developed using **C++** for the application layer and **MySQL** for database management. It provides secure user authentication, tour package management, customer booking, hotel and transport reservation, payment management, customer reviews, notification services, refund processing, emergency contacts, and administrative reporting.

Advanced database concepts such as **Views, Triggers, Stored Procedures, Indexes, Aggregate Queries, and Relational Database Design** have been implemented to improve efficiency, maintainability, and query performance. The database has been designed following normalization principles to minimize redundancy and maintain data integrity.

This project demonstrates practical implementation of Database Management System concepts in a real-world scenario and provides a scalable foundation for future enhancements such as online payment gateways, mobile applications, GPS-based tour tracking, and AI-powered travel recommendations.

INTRODUCTION

1.1 Introduction :

Tourism is one of the fastest-growing industries in the world, contributing significantly to economic development and employment opportunities. As the number of travelers increases, the demand for efficient and automated tour management systems has also grown. Traditional tour management processes often rely on manual record-keeping, paperwork, and human coordination, which can lead to delays, errors, and data inconsistency.

The **Tour Management System** is a database-driven software application developed to automate and simplify the complete tour management process. The system provides a centralized platform where administrators can manage tour packages, customers, bookings, hotels, transportation, payments, reviews, and reports efficiently.

This project has been developed using **C++** for the frontend application and **MySQL** as the backend relational database. It demonstrates the practical implementation of Database Management System concepts such as normalization, primary keys, foreign keys, views, triggers, stored procedures, indexing, joins, and aggregate functions.

The main objective of this project is to reduce manual work, improve data accuracy, increase operational efficiency, and provide better customer service through a secure and well-organized database system.

1.2 Background of the Study

The tourism industry requires effective management of a large amount of information including customer records, tour packages, hotel reservations, transportation schedules, payment transactions, employee details, and customer feedback.

Many small and medium-sized travel agencies still maintain these records manually using notebooks or spreadsheets. Manual systems consume more time, increase the possibility of human errors, and make data retrieval difficult.

Modern database systems solve these problems by providing secure storage, quick searching, efficient data processing, and automated operations. The Tour Management System has been designed to replace manual operations with an intelligent computerized solution capable of handling all major tour-related activities from a single integrated platform.

1.3 Problem Statement

Many travel agencies continue to use traditional management methods that create several operational challenges.

The major problems are:

- Manual record keeping increases the possibility of human error.
- Searching customer or booking information requires significant time.
- Maintaining payment records manually is difficult.
- Hotel and transport management become complicated when bookings increase.
- There is no automatic notification system.
- Data duplication reduces database efficiency.
- Generating reports manually requires additional effort.
- Managing customer reviews and ratings becomes difficult.
- Security of customer information cannot be guaranteed.
- Lack of centralized data management causes inconsistency.

These problems reduce overall productivity and customer satisfaction.

1.4 Proposed Solution

To overcome the limitations of manual tour management, this project introduces a complete **Database Management System** that automates all major business operations.

The proposed system provides:

- Secure User Authentication
- Customer Registration
- Tour Package Management
- Online Booking Management
- Hotel Reservation
- Transport Reservation
- Payment Management
- Customer Reviews
- Notifications using Database Trigger
- Stored Procedures for Booking History
- Database Views for Secure Reporting
- Database Indexing for Faster Query Execution
- Administrative Dashboard
- Statistical Reports
- Refund Management
- Emergency Contact Management
- Support Ticket Management

The proposed solution significantly improves efficiency, security, and maintainability.

1.5 Objectives of the Project

The primary objectives of the Tour Management System are:

General Objectives

- To develop a complete Tour Management System using C++ and MySQL.
- To implement relational database concepts in a real-world application.
- To automate booking and payment processes.
- To improve data consistency and integrity.

Specific Objectives

- Develop a secure login system.
- Manage customer information.
- Maintain tour package records.
- Process customer bookings.
- Manage hotel reservations.
- Manage transportation services.
- Record payment transactions.
- Generate statistical reports.
- Store customer reviews.
- Implement notification system using triggers.
- Implement stored procedures.
- Implement database views.
- Improve query performance using indexes.
- Reduce data redundancy through normalization.

1.6 Scope of the Project

The scope of this project includes the complete management of tour-related business activities.

The system supports:

- User Registration
- Login Authentication
- Tour Package Management
- Destination Management
- Booking Management
- Hotel Booking

- Transport Booking
- Payment Processing
- Customer Reviews
- Wishlist
- Coupons
- Tour Guides
- Emergency Contacts
- Notifications
- Support Tickets
- Refund Processing
- Expense Management
- Statistical Analysis
- Administrative Reports

The system is designed for educational purposes but can be extended into a commercial application with additional features.

1.7 Significance of the Project

This project is significant because it demonstrates practical implementation of Database Management System concepts learned during academic study.

The system provides several important benefits:

- Reduces manual work.
- Improves operational efficiency.
- Ensures data integrity.
- Provides secure database management.
- Enhances customer satisfaction.
- Generates reports quickly.
- Improves query performance using indexes.
- Demonstrates practical implementation of SQL.
- Applies advanced database concepts.
- Serves as a learning platform for database development.

1.8 Project Features

The major features implemented in this project are:

- User Authentication
- Admin Dashboard
- Customer Dashboard

- Tour Package Management
- Destination Management
- Booking System
- Hotel Reservation
- Transport Reservation
- Payment Module
- Review & Rating System
- Notification System
- Wishlist
- Coupons
- Tour Guides
- Emergency Contacts
- Support Ticket System
- Refund Management
- Expense Management
- Statistical Reports
- Database Views
- Database Triggers
- Stored Procedures
- Database Indexing
- Aggregate SQL Queries

1.9 Development Tools

The following tools and technologies were used during the development of this project.

Component	Technology Used
Programming Language	C++
Database	MySQL 8.0
IDE	Code::Blocks
Database Tool	MySQL Workbench
Version Control	GitHub
Operating System	Windows 11
Documentation	Microsoft Word
Report Format	PDF

SYSTEM ANALYSIS AND REQUIREMENT ANALYSIS

2.1 System Analysis

System analysis is one of the most important phases of software development. It involves studying the existing system, identifying its limitations, gathering user requirements, and designing an improved solution. A proper system analysis helps developers understand business processes and ensures that the final software meets user expectations.

For this project, a detailed analysis was conducted to understand how travel agencies currently manage customer information, bookings, hotel reservations, transportation services, payments, and tour packages. The study revealed several operational problems that can be solved through a computerized database management system.

The Tour Management System has been designed to provide a secure, efficient, and centralized solution that automates all major tour management activities while ensuring data consistency and integrity.

2.2 Existing System

Many travel agencies still use manual systems or basic spreadsheet applications to manage their daily operations. These traditional systems suffer from several disadvantages.

Problems of Existing System

- Manual customer registration
- Paper-based booking records
- Time-consuming payment management
- Duplicate customer information
- Slow searching process
- Poor report generation
- High probability of human errors
- Lack of security
- No automatic notification system
- Difficult hotel and transport management
- Inefficient customer review management
- Poor data consistency

These limitations reduce overall organizational productivity and customer satisfaction.

2.3 Proposed System

The proposed Tour Management System replaces manual operations with a fully computerized database-driven solution.

The proposed system allows administrators to manage all tour-related information through a centralized database while customers can securely access tour packages, make bookings, submit reviews, and manage payments.

The system integrates multiple modules into a single platform, making tour management easier, faster, and more reliable.

2.4 Advantages of the Proposed System

The proposed system offers numerous advantages over the traditional manual system.

Major Advantages

- Faster customer registration
- Secure login authentication
- Easy package management
- Automated booking process
- Hotel reservation management
- Transport management
- Secure payment records
- Customer review management
- Automatic notifications using database triggers
- Booking history through stored procedures
- Faster searching using database indexing
- Better report generation
- Reduced paperwork
- Improved data integrity
- Better customer service
- High database security
- Easy maintenance
- Scalability for future development

2.5 Feasibility Study

A feasibility study determines whether the project can be successfully implemented.

2.5.1 Technical Feasibility

The project has been developed using technologies that are widely available and easy to implement.

Technologies Used

- C++
- MySQL Server
- MySQL Workbench
- Code::Blocks IDE
- Windows Operating System
- GitHub

The required software tools are freely available, making the project technically feasible.

2.5.2 Economic Feasibility

The project requires very low development cost because all development tools are free.

There are no expensive software licenses required.

Therefore, the system is economically feasible for educational institutions and small travel agencies.

2.5.3 Operational Feasibility

The system has been designed with a simple and user-friendly interface.

Both administrators and customers can easily operate the software without requiring advanced technical knowledge.

2.5.4 Schedule Feasibility

The project can be completed within the allocated academic semester.

The development process includes planning, database design, coding, testing, documentation, and deployment.

2.6 Functional Requirements

Functional requirements describe what the system should do.

User Module

- User Registration
- User Login
- Password Verification
- Profile Management

Tour Package Module

- Add Package
- Update Package
- Delete Package
- Search Package
- View Package Details

Booking Module

- Book Tour
- Cancel Booking
- View Booking History
- Booking Confirmation

Hotel Module

- Add Hotel
- Book Hotel
- Check Room Availability

Transport Module

- Manage Transport
- Reserve Seat
- View Transport Information

Payment Module

- Payment Processing
- Payment History
- Payment Status

Review Module

- Submit Review
- Rate Tour Package
- View Reviews

Notification Module

- Booking Notification
- Payment Notification
- Trigger-based Notification

Report Module

- Booking Reports
- Revenue Reports
- Customer Reports
- Statistics

2.7 Non-Functional Requirements

Non-functional requirements define the quality attributes of the software.

Performance

The system should provide fast query execution through database indexing.

Reliability

The system should maintain accurate and consistent data.

Security

Only authenticated users can access the system.

Availability

The database should remain available whenever users need it.

Scalability

New modules can easily be added in the future.

Maintainability

The software should be easy to update and maintain.

Portability

The system can be executed on any Windows computer with MySQL installed.

Usability

The user interface should be simple and easy to understand.

2.8 Software Requirements

The software requirements are listed below.

Software	Version
Windows 11	Latest
Code::Blocks	Latest
MySQL Server	8.0
MySQL Workbench	Latest
Git	Latest
GitHub	Online

Microsoft Word	2021

2.9 Hardware Requirements

Hardware	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or Higher
RAM	Minimum 8 GB
Storage	10 GB Free Space
Display	1366×768 or Higher
Internet	Required for GitHub

2.10 Development Methodology

The project follows the **Waterfall Software Development Model**.

The development process includes:

1. Requirement Analysis
2. System Design
3. Database Design
4. Coding
5. Testing
6. Documentation
7. Deployment

Each phase was completed before moving to the next stage, ensuring systematic project development.

2.11 System Workflow

The workflow of the Tour Management System is as follows:

1. User logs into the system.
2. System verifies login credentials.
3. Customer views available tour packages.

4. Customer selects a package.
5. Booking information is stored in the database.
6. Trigger automatically generates a notification.
7. Payment is processed.
8. Booking status is updated.
9. Customer submits review after completing the tour.
10. Administrator generates reports and statistics.

SYSTEM DESIGN

3.1 Introduction

System design is one of the most important phases of software development. It defines the overall architecture, database structure, relationships between entities, and the interaction among different modules. A well-designed system ensures better performance, maintainability, scalability, and security.

The **Tour Management System** has been designed using a relational database model where all data is stored in normalized tables. The frontend application has been developed using **C++**, while **MySQL** is used as the backend database management system.

The system architecture consists of multiple interconnected modules such as user management, booking management, hotel management, transport management, payment management, notification management, review management, and report generation.

3.2 System Architecture

The overall architecture of the Tour Management System follows a **Three-Tier Architecture**.

Presentation Layer

The Presentation Layer is developed using **C++ Console Application**. It provides a user-friendly interface where administrators and customers can perform different operations such as login, booking, payment, and report generation.

Business Logic Layer

This layer contains all the application logic including authentication, booking validation, payment calculation, notification generation, and report processing.

Database Layer

The Database Layer is implemented using **MySQL**. It stores all information related to users, tour packages, bookings, hotels, payments, reviews, transport, notifications, and other system data.

Figure 3.1 System Architecture

> **Insert Figure Here**

Image: `System_Architecture.png`

Figure 3.1: Overall Architecture of the Tour Management System.

3.3 Prisma Diagram

The Prisma Diagram illustrates the interaction among different functional modules of the Tour Management System.

The system starts from the user authentication module. After successful login, customers can browse tour packages, make bookings, reserve hotels and transportation, complete payments, submit reviews, and receive notifications. Administrators can manage all records and generate reports.

Figure 3.2 Prisma Diagram

> **Insert Figure Here**

Image: `Prisma_Diagram.png`

Figure 3.2: Prisma Diagram of the Tour Management System.

3.4 Entity Relationship (ER) Diagram

The Entity Relationship Diagram (ERD) represents the logical relationship among all database entities.

The database consists of several interconnected entities including:

- Users
- Tour Packages
- Destinations
- Bookings
- Payments
- Hotels
- Hotel Booking
- Transport
- Transport Booking
- Reviews
- Notifications
- Tour Guides
- Employees
- Refunds
- Support Tickets
- Activities
- Coupons
- Wishlist
- Emergency Contacts
- Insurance
- Expenses

Each entity is connected through **Primary Keys** and **Foreign Keys**, ensuring referential integrity.

Figure 3.3 ER Diagram

> **Insert Figure Here**

Image: `ER_Diagram.png`

Figure 3.3: Entity Relationship Diagram of the Tour Management System.

3.5 Relational Schema

The Relational Schema represents the database tables together with their primary keys and foreign key relationships.

The database has been normalized to reduce redundancy and improve consistency.

Each table has a unique primary key, and relationships are maintained using foreign keys.

Figure 3.4 Relational Schema

Insert Figure Here

Image: `Relational_Schema.png`

Figure 3.4: Relational Schema of the Tour Management System.

3.6 Database Design

The Tour Management System database has been designed following the principles of relational database management systems.

The database includes more than **20 interconnected tables**.

Major entities are:

- Users
- Destinations
- TourPackages
- Bookings
- Payments
- Hotels
- HotelBooking
- Transport
- TransportBooking
- Employees
- Reviews
- Coupons
- Wishlist
- Notifications
- TourGuides
- PackageGuide
- EmergencyContacts
- Insurance
- Expenses
- Activities
- PackageActivities
- Refunds

- SupportTickets

The database uses:

- Primary Keys
- Foreign Keys
- Unique Constraints
- CHECK Constraints
- Indexes
- Views
- Triggers
- Stored Procedures

to maintain consistency and improve performance.

3.7 Database Tables Description

Users

Stores administrator and customer information including authentication credentials.

Primary Key: user_id

Destinations

Stores available travel destinations.

Primary Key: destination_id

TourPackages

Contains package details including package name, destination, duration, price, and maximum number of tourists.

Primary Key: package_id

Foreign Key:

- created_by → Users

PackageDestination

Represents the many-to-many relationship between packages and destinations.

Bookings

Stores customer booking information.

Fields include:

- Booking Date
- Travel Date
- Total People
- Total Cost
- Booking Status

Foreign Keys:

- user_id
- package_id

Payments

Stores payment records.

Fields include:

- Payment Method
- Amount
- Payment Date
- Payment Status

Hotels

Stores hotel information.

Fields include:

- Hotel Name
- Location
- Rating
- Available Rooms

HotelBooking

Maintains hotel reservation records.

Transport

Stores transport information.

TransportBooking

Stores transport reservation details.

Employees

Maintains employee information.

Reviews

Stores customer ratings and comments.

Uses CHECK Constraint:

Rating BETWEEN 1 AND 5

Coupons

Stores promotional coupon information.

Wishlist

Allows customers to save favorite tour packages.

Notifications

Stores automatic notifications generated through database triggers.

TourGuides

Stores tour guide information.

PackageGuide

Connects tour guides with packages.

EmergencyContacts

Stores emergency contact information.

Insurance

Stores travel insurance records.

Expenses

Stores company expenses.

Activities

Stores activity information included in tour packages.

PackageActivities

Represents package-activity relationships.

Refunds

Stores refund information.

SupportTickets

Stores customer support requests.

3.8 Database Relationships

The following relationships exist within the database:

- One User can create many Bookings.
- One Package can have many Bookings.
- One Booking can have one Payment.
- One Booking can reserve one Hotel.
- One Booking can reserve one Transport.
- One Package can receive many Reviews.
- One User can submit multiple Reviews.
- One User can receive multiple Notifications.
- One Package can include multiple Activities.
- One Package can have multiple Tour Guides.

These relationships ensure proper normalization and reduce data redundancy.

3.9 Normalization

The database has been normalized up to the **Third Normal Form (3NF)**.

First Normal Form (1NF)

- Atomic values
- No repeating groups

Second Normal Form (2NF)

- Full dependency on primary keys
- Partial dependency removed

Third Normal Form (3NF)

- Transitive dependencies removed
- Better data consistency
- Reduced redundancy

Normalization improves database efficiency and minimizes anomalies.

SYSTEM IMPLEMENTATION

4.1 Introduction

System implementation is the phase where the designed system is transformed into a fully functional software application. In this project, the Tour Management System has been implemented using **C++** for the frontend application and **MySQL** as the backend relational database. The implementation integrates object-oriented programming concepts with relational database management techniques to create an efficient and user-friendly system.

The application allows administrators and customers to perform different operations such as user authentication, package management, booking, payment processing, hotel reservation, transport reservation, customer review management, and report generation.

4.2 Development Environment

The following software and hardware environment was used during the implementation of the project.

Component	Technology Used
Programming Language	C++
Database	MySQL 8.0
IDE	Code::Blocks
Database Tool	MySQL Workbench
Version Control	GitHub
Operating System	Windows 11

4.3 Programming Language (C++)

The frontend application was developed using **C++**, which follows the Object-Oriented Programming (OOP) paradigm. The implementation uses structures, vectors, functions, loops, conditional statements, and file handling techniques to simulate database operations and interact with MySQL.

The application is menu-driven, making it simple and easy for both administrators and customers to use.

4.4 Database Implementation (MySQL)

The backend database has been implemented using **MySQL**. All project data including users, bookings, payments, hotels, transport, reviews, notifications, and reports are stored in relational tables.

The database contains more than **20 normalized tables**, connected through primary and foreign key relationships.

4.5 Login Module

The Login Module authenticates users before allowing access to the system.

Features

- Secure Login
- Email Verification
- Password Verification
- Admin Login
- Customer Login
- Session Management

After successful authentication, the system redirects users to the appropriate dashboard based on their role.

Figure 4.1 Login Interface

Insert Screenshot Here

Image:

`login.png`

4.6 Admin Module

The Admin Module provides complete control over the Tour Management System.

The administrator can perform the following operations:

- Add New Tour Package
- View Customer Bookings
- Manage Tour Packages
- Monitor Notifications
- View Statistics
- Manage Reports

The administrator can efficiently monitor the entire system from a single dashboard.

Figure 4.2 Admin Dashboard

Insert Screenshot Here

Image:

`admin_menu.png`

4.7 Customer Module

The Customer Module allows registered users to interact with the Tour Management System.

Customers can:

- Login
- View Tour Packages
- Book Tour Packages
- View Booking History
- View Notifications
- Submit Reviews

The customer dashboard provides a simple and user-friendly interface.

Figure 4.3 Customer Dashboard

Insert Screenshot Here

Image:

`customer_menu.png`

4.8 Booking Module

The Booking Module is responsible for handling customer reservations.

During booking, the customer selects a tour package, enters the travel date, specifies the number of people, and confirms the reservation.

The system automatically calculates the total cost and stores the booking information in the database.

Once a booking is successfully completed, a database trigger generates a notification automatically.

Figure 4.4 Booking Module

Insert Screenshot Here

Image:

`booking.png`

4.9 Payment Module

The Payment Module manages all financial transactions related to tour bookings.

The module stores:

- Booking ID
- Payment Method
- Payment Amount
- Payment Date
- Payment Status

It maintains accurate payment records for future reporting.

Figure 4.5 Payment Module

Insert Screenshot Here

Image:

`payment.png`

4.10 Statistics Module

The Statistics Module generates useful business reports using SQL aggregate functions.

Implemented reports include:

- Total Revenue
- Total Bookings
- Average Ratings
- Total Customer Spending
- Hotel Availability
- Popular Tour Packages

These reports help administrators make better business decisions.

Figure 4.6 Statistics Dashboard

Insert Screenshot Here

Image:

`statistics.png`

4.11 Database Trigger

A database trigger named **booking_notification** has been implemented.

Whenever a new booking is inserted into the **Bookings** table, the trigger automatically inserts a notification into the **Notifications** table.

This automation improves system efficiency and reduces manual work.

Example:

```
CREATE TRIGGER booking_notification
AFTER INSERT ON Bookings
FOR EACH ROW
```

4.12 Stored Procedure

The project implements a stored procedure named **GetUserBookings()**.

The procedure retrieves the booking history of a specific customer based on the user ID.

Advantages:

- Faster execution
- Improved security
- Reusable SQL code
- Reduced query complexity

4.13 Database View

A database view named **CustomerBookingView** has been created.

The view joins multiple tables to display customer booking information without exposing unnecessary database details.

Benefits:

- Simplified queries
- Enhanced security
- Improved readability

4.14 Database Indexing

Two indexes have been implemented to improve query performance.

- idx_destination
- idx_booking_date

These indexes significantly reduce query execution time when searching large datasets.

4.15 SQL Features Implemented

The project successfully implements the following SQL concepts:

- Primary Key
- Foreign Key
- Auto Increment
- CHECK Constraint
- JOIN Operations
- GROUP BY
- ORDER BY
- Aggregate Functions
- Nested Query
- View
- Trigger
- Stored Procedure
- Index
- INSERT
- UPDATE
- DELETE
- SELECT

4.16 C++ Features Implemented

The C++ implementation includes several programming concepts.

Object-Oriented Programming Features

- Structures (struct)
- Functions
- Vectors

- Conditional Statements
- Loops
- Menu-driven Programming
- Modular Design

Standard Libraries Used

- iostream
- string
- vector
- fstream
- iomanip

These libraries provide input/output operations, data storage, formatting, and file handling capabilities.

SYSTEM TESTING AND RESULTS

5.1 Introduction

System testing is a crucial phase of software development that verifies whether the developed application satisfies the specified functional and non-functional requirements. The Tour Management System was tested using different test cases to ensure that every module works correctly and produces accurate results.

The testing process included login verification, package management, booking operations, payment processing, trigger execution, stored procedure execution, report generation, and database performance analysis.

5.2 Testing Objectives

The objectives of system testing are:

- Verify all system functionalities.
- Detect logical and runtime errors.
- Validate database operations.
- Ensure data consistency.
- Evaluate system performance.

- Verify SQL query execution.
- Ensure user authentication works correctly.
- Test trigger, view, and stored procedure functionality.

5.3 Testing Environment

Component	Details
Operating System	Windows 11
IDE	Code::Blocks
Programming Language	C++
Database	MySQL Server 8.0
Database Tool	MySQL Workbench
Version Control	GitHub

5.4 Test Cases

Table 5.1 Functional Test Cases

Test ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	Login with valid credentials	Login successful	Successful	✅ Pass
TC-02	Login with invalid credentials	Error message	Displayed	✅ Pass
TC-03	View tour packages	Package list displayed	Successful	✅ Pass
TC-04	Book a tour package	Booking created	Successful	✅ Pass
TC-05	Trigger execution	Notification inserted	Successful	✅ Pass

TC-06	View booking history	Booking displayed	Successful	✅ Pass
TC-07	Add new package (Admin)	Package added	Successful	✅ Pass
TC-08	View notifications	Notification list displayed	Successful	✅ Pass
TC-09	Execute stored procedure	Booking history returned	Successful	✅ Pass
TC-10	Execute view	Booking information displayed	Successful	✅ Pass
TC-11	Payment processing	Payment stored	Successful	✅ Pass
TC-12	Statistics generation	Reports generated	Successful	✅ Pass

---

5.5 Output Screenshots

The following screenshots demonstrate the successful execution of different modules of the Tour Management System.

Figure 5.1 Login Screen

Insert Screenshot

Image:

`login.png`

Description:

The login screen authenticates administrators and customers using their email address and password before granting access to the system.

Figure 5.2 Customer Dashboard

Insert Screenshot

Image:

`customer_menu.png`

Description:

The customer dashboard allows users to view available tour packages, book tours, and access booking history.

Figure 5.3 Admin Dashboard

Insert Screenshot

Image:

`admin_menu.png`

Description:

The administrator dashboard provides complete control over package management, bookings, reports, and notifications.

Figure 5.4 Booking Module

Insert Screenshot

Image:

`booking.png`

Description:

The booking module allows customers to reserve tour packages and automatically calculates the total booking cost.

Figure 5.5 Payment Module

Insert Screenshot

Image:

`payment.png`

Description:

The payment module stores payment information and maintains transaction history.

Figure 5.6 Statistics Dashboard

Insert Screenshot

Image:

`statistics.png`

Description:

The statistics dashboard generates business reports such as total revenue, bookings, customer spending, and average ratings.

5.6 SQL Query Results

Several SQL queries were executed successfully during testing.

Total Revenue

```
SELECT SUM(amount) AS Total_Revenue
FROM Payments
WHERE payment_status='Paid';
```

Purpose:

Calculates the total revenue generated from completed payments.

Total Bookings

```
SELECT package_name,
COUNT(booking_id)
FROM Bookings
JOIN TourPackages
ON Bookings.package_id=TourPackages.package_id
GROUP BY package_name;
```

Purpose:

Displays the total number of bookings for each tour package.

Average Rating

```
SELECT package_name,
AVG(rating)
FROM Reviews
JOIN TourPackages
ON Reviews.package_id=TourPackages.package_id
GROUP BY package_name;
```

Purpose:

Calculates the average customer rating for each tour package.

Customer Booking View

```
SELECT *
FROM CustomerBookingView;
```

Purpose:

Displays booking information using the database view.

Stored Procedure

```
CALL GetUserBookings(2);
```

Purpose:

Retrieves the booking history of User ID 2.

5.7 Trigger Testing

The **booking_notification** trigger was tested by inserting a new booking into the **Bookings** table.

Result:

- Booking inserted successfully.

- Notification automatically generated.
- Notification stored in the Notifications table.

Status:

✅ Passed

5.8 Performance Analysis

Database indexing significantly improved query execution performance.

Implemented indexes:

- idx_destination
- idx_booking_date

Advantages:

- Faster searching
- Reduced query execution time
- Improved database efficiency
- Better scalability

5.9 Advantages of the System

The developed Tour Management System provides several benefits.

- User-friendly interface
- Secure authentication
- Fast booking process
- Automatic notifications
- Efficient payment management
- Reduced manual work
- Centralized database
- High data integrity
- Better reporting
- Improved query performance
- Easy maintenance
- Scalable architecture

5.10 Limitations

Although the system performs efficiently, some limitations remain.

- Console-based user interface
- Online payment gateway is not implemented
- Email notification service is unavailable
- Mobile application is not available
- GPS tracking is not included
- Multi-language support is not implemented

5.11 Future Improvements

The following enhancements can be implemented in future versions.

- Web-based application
- Android application
- Online payment integration
- QR code ticket generation
- AI-based tour recommendation
- GPS tracking
- Cloud database
- Real-time notification
- Online chat support
- Machine Learning-based recommendation system

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The **Tour Management System** is a complete database-driven application designed to simplify and automate tour management activities. The project successfully demonstrates the practical implementation of **Database Management System (DBMS)** concepts using **MySQL** and **C++**.

The developed system efficiently manages users, tour packages, destinations, bookings, hotels, transportation, payments, reviews, notifications, refunds, expenses, and reports through a centralized database. By replacing manual operations with an automated solution, the system significantly improves efficiency, reduces human errors, and maintains data consistency.

Advanced database features such as **Primary Keys, Foreign Keys, Views, Stored Procedures, Triggers, Indexes, Aggregate Functions, and SQL Joins** have been successfully implemented. These features improve database performance, ensure referential integrity, and simplify complex operations.

The frontend application has been developed using **C++** with a menu-driven interface, while **MySQL** provides secure and reliable data storage. The project follows proper relational database design principles and normalization techniques to eliminate redundancy and maintain data integrity.

Overall, this project successfully fulfills the objectives of the Database Management Systems Lab course and demonstrates the practical application of theoretical database concepts in a real-world tour management scenario.

6.2 Project Achievements

During the development of this project, the following objectives were successfully achieved:

- Developed a complete Tour Management System using C++ and MySQL.
- Designed a normalized relational database.
- Implemented secure login authentication.
- Managed customer and administrator information.
- Developed tour package management.
- Implemented booking management.
- Developed hotel booking and transport booking modules.
- Implemented payment management.
- Developed review and rating system.
- Implemented wishlist and coupon management.
- Created automatic notification system using database triggers.
- Implemented stored procedures for booking history.
- Developed database views for secure reporting.
- Improved query performance using indexing.
- Generated statistical reports using SQL aggregate functions.
- Maintained referential integrity using foreign keys.
- Successfully tested all modules.

6.3 Learning Outcomes

This project provided practical experience in various database and software development concepts.

The major learning outcomes include:

- Relational Database Design
- Database Normalization
- SQL Programming
- Primary & Foreign Keys
- Database Constraints
- Database Indexing
- Views
- Stored Procedures
- Triggers
- Aggregate Functions
- SQL Joins
- Database Optimization
- Object-Oriented Programming
- C++ Programming
- MySQL Database Administration
- GitHub Project Management
- Software Documentation

The project enhanced both programming skills and database management knowledge.

6.4 Future Scope

Although the current system performs efficiently, several advanced features can be added in future versions.

Future Improvements

- Web-based application using Laravel or ASP.NET
- Android Mobile Application
- Online Payment Gateway Integration
- Email Notification Service
- SMS Notification
- QR Code Ticket Generation
- GPS-based Tour Tracking
- AI-based Tour Recommendation
- Machine Learning Recommendation Engine
- Cloud Database Integration

- Online Hotel API Integration
- Online Flight Ticket Booking
- Google Maps Integration
- Multi-language Support
- Live Chat Support
- Admin Analytics Dashboard
- Online Invoice Generation
- Customer Loyalty Program
- Digital Wallet Support
- Facial Recognition Login

These features will transform the project into a commercial-level Tour Management System.

6.5 References

The following resources were used during the development of this project.

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*. Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill.
3. Oracle Corporation. *MySQL 8.0 Reference Manual.*
4. Bjarne Stroustrup. *The C++ Programming Language.*
5. ISO/IEC. *Programming Language C++ Standard.*
6. GitHub Documentation.
7. MySQL Workbench Documentation.
8. Code::Blocks Official Documentation.
9. International Islamic University Chittagong (IIUC) DBMS Lab Course Materials.

6.6 Appendix

The following project files are included with the submission.

Source Code

- main.cpp

Database Script

- tour_Management.sql

Documentation

- Project_Report.pdf

GitHub Repository

- README.md
- LICENSE

Database Design

- Prisma_Diagram.png
- ER_Diagram.png
- Relational_Schema.png

Screenshots

- login.png
- admin_menu.png
- customer_menu.png
- booking.png
- payment.png
- statistics.png

6.7 Declaration

I hereby declare that this project entitled **"Tour Management System"** is my original work completed as part of the requirements for the course **Database Management Systems Lab (CSE 2424)** under the supervision of **Mr. Mizanur Rahman**.

All sources of information used in this report have been properly acknowledged and referenced. I further declare that this project has not been submitted to any other institution for academic credit.

Student Name: Md. Motiur Rahman

Student ID: C243119

Department: Computer Science & Engineering

International Islamic University Chittagong (IIUC)

Signature: _____

Date: _____

6.8 Final Summary

The Tour Management System successfully integrates database concepts with software development principles to provide a complete tour management solution. The implementation of relational database design, normalization, SQL programming, triggers, views, stored procedures, indexing, and object-oriented programming demonstrates a strong understanding of Database Management Systems. This project not only satisfies the academic requirements of the course but also provides a scalable foundation for future real-world development.